

COMPSYS 723

Assignment 1: Low Cost Frequency Relay (LCFR)

Author: Christian Motta and Tania Pan

Date: 2026-04-20

1 Introduction

The stability of a power grid relies on maintaining a strict balance between power generation and load consumption. In New Zealand, this nominal operating frequency is 50 Hz. When faults occur (such as generator failure), the frequency drops rapidly. To prevent cascading failures, a Low Cost Frequency Relay (LCFR) is deployed to detect these anomalies and shed loads within strict timing constraints (200ms).

This report details the design and implementation of an LCFR system on an Altera DE2-115 board using a Nios II soft core processor. The system utilizes FreeRTOS to manage concurrent tasks, ensuring real time constraints are met for frequency analysis and load shedding, while simultaneously providing a responsive interface via VGA and a PS/2 keyboard.

2 System Architecture and Task Allocation

The system functionality is split into specific hardware Interrupt Service Routines (ISRs) and software FreeRTOS tasks to ensure modularity and responsiveness.

2.1 Interrupt Service Routines (ISRs)

- **Frequency Analysis Unit (FAU) ISR:** Triggers when a new peak is detected by the ADC. It reads the sample count, clears the interrupt, and gives a semaphore to unblock the frequency calculation task.
- **Push Button ISR:** Captures physical button presses for toggling the maintenance state and sends the command to a queue.
- **PS/2 Keyboard ISR:** Captures keyboard codes and sends them to a queue.

2.2 FreeRTOS Tasks

The software is split into three primary tasks based on their specific roles:

1. **Frequency Calculation:** A high priority task that waits for the FAU semaphore, calculates the current frequency and Rate of Change (RoC), and passes the data forward.
2. **Load Management (Relay Logic):** Acts as the decision maker, evaluating the incoming frequency data against the dynamic thresholds (T_F and T_{ROCF}). It manages the timers for the 500ms shedding/reconnecting logic, maps load statuses to LEDs, and polls physical wall switches.
3. **VGA Display (VGA & Keyboard):** A low priority task that parses keyboard inputs to adjust thresholds and updates the VGA display with system states.

3 Inter Process Communication

We utilized a mix of queues, semaphores, and mutexes based on the nature of the data being shared.

3.1 Queues (freq_data_q, kbd_q, button_q)

Queues were chosen for streams of events where discrete event mattered and should be processed sequentially.

- **Keyboard and Buttons:** User inputs happen asynchronously. A queue ensures that rapid, successive key presses or button toggles are buffered and processed in order without dropping events.
- **Frequency Data:** The analyzer pushes calculated frequency/RoC structs to `freq_data_q`. The Decision Maker must evaluate every single reading to accurately detect threshold breaches within the 200ms window.

3.2 Mutexes (`load_status`, `system_status`, `timing_log`)

Mutexes were chosen over queues for variables that represent the current state of the system.

- For the VGA task to display the `load_status` or the current TF threshold, it does not need a chronological history of every time the state changed. It only requires safe read access to the most up to date value.
- Mutexes prevent race conditions (e.g. the display reading the timing logs while the load manager is in the middle of updating the logs) without the memory overhead of queuing large structs.

3.3 Binary Semaphores (`peak_ready_sem`)

A binary semaphore is used for synchronization between the FAU ISR and the Frequency Calculation task. This acts as a pure signaling mechanism (“data is ready”), allowing the analyzer task to remain blocked (consuming zero CPU cycles) until the hardware signals a new peak.

4 Task Priorities and Scheduling

FreeRTOS uses a preemptive priority based scheduler. Priorities were assigned based on tasks with the strictest deadlines receiving the highest priorities.

- **Priority 3 (High) - Frequency Calculation:** Calculating the frequency and RoC is the most time critical software operation. If this task is delayed, the system cannot detect faults, leading to failure to shed loads within 200ms.
- **Priority 2 (Medium) - Load Management:** This task depends on the data from the analyzer. It must preempt the display task to execute shedding logic and timer updates immediately after an anomaly is detected.
- **Priority 1 (Low) - VGA Display:** Updating the VGA and processing keyboard inputs has no strict real time deadline regarding grid stability. Running this at the lowest priority ensures that human interface updates never starve the critical relay protection logic.

5 Periodic vs. Event Driven Tasks

- **Event Driven Tasks:** The frequency calculation and load management tasks are entirely event driven. They do not run on a fixed schedule. Instead, they block on semaphores or queues, executing only when new hardware data arrives or when an anomaly requires action. This maximizes CPU availability.
- **Periodic (Time-Driven) Tasks:** The VGA Display task is periodic. It is designed with a `vTaskDelayUntil` loop to run at approximately 60Hz (every 17ms). This provides a smooth visual refresh rate for the monitor while explicitly yielding the CPU back to the scheduler for the remainder of the 17ms window. Additionally, the load management shedding/reconnection sequence utilizes a 500ms periodic software timer to ensure consecutive loads are staggered properly.

6 System Performance & Real-Time Verification

A core requirement of the LCFR is to shed the first load within 200ms of detecting an unstable network state. We measured this by recording the FreeRTOS tick count at the exact moment a threshold violation was calculated, and comparing it to the tick count right after the first load status was updated to `LOAD_SHED`.

During final testing, our system recorded response times between 0ms and 1ms (averaging 0ms), easily meeting the 200ms requirement. To verify that our timing logic was actually working and not just outputting zeros, we temporarily injected a 50ms forced delay loop into the shedding logic. As expected, the VGA display updated to show minimum, maximum, and average response times hovering precisely around 50ms.



Figure 1: VGA display showing normal operation with 0-1ms response times.

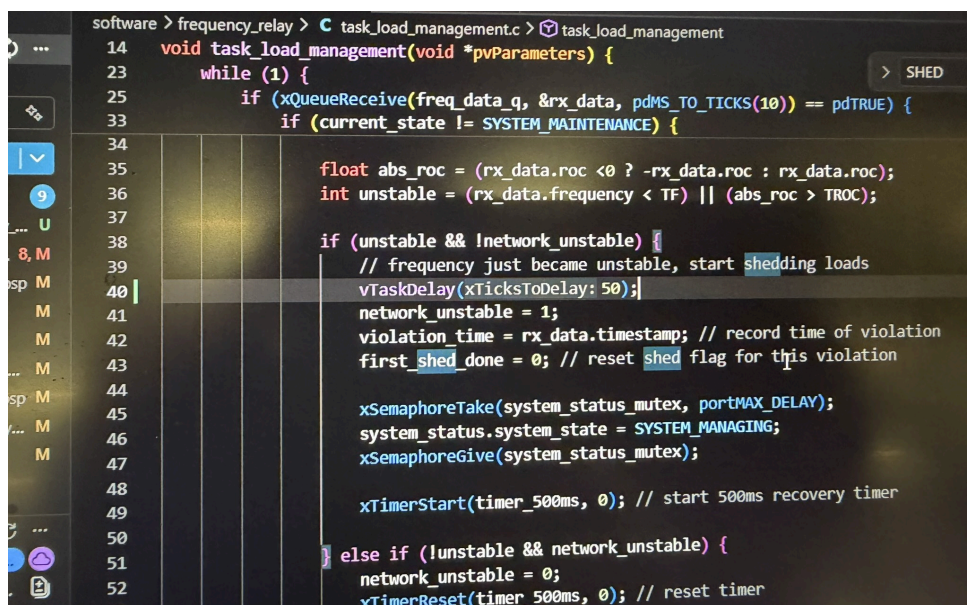


Figure 2: Code modification with forced 50ms delay.

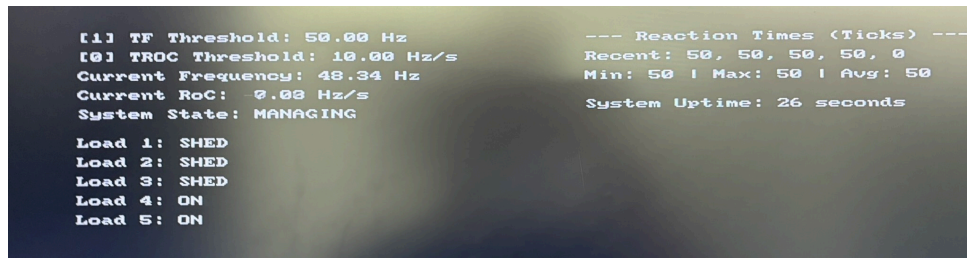


Figure 3: VGA display showing the verified timing logic with a forced 50ms delay.

7 Challenges and Design Decisions

During development, we ran into several RTOS concurrency and hardware integration issues that required specific design decisions to fix.

- **Mutex Deadlocks:** Initially, our response times spiked to over 2000ms. We realized this was caused by self deadlocking. Tasks were attempting to take a mutex they already held, or getting stuck waiting for a lock while nested functions were blocking each other. We resolved this by restructuring our code to release mutexes earlier and preventing operations like LED updates from holding the system status lock unnecessarily.
- **Queue Starvation:** Originally, the VGA task tried to peek at `freq_data_q` to display the current frequency. However, the higher priority load management task was consuming all queue items before the VGA task could run. We fixed this by having the frequency calculation task copy the newest data into a separate global struct (`ui_freq_data`) protected by its own mutex, leaving the queue purely for the logic task.
- **VGA Rendering Timings:** We initially used `vTaskDelay` to regulate the VGA task to 60Hz. This caused flickering because it didn't account for the time spent executing the text buffer writes. Switching to `vTaskDelayUntil` fixed this by establishing an absolute 17ms interval regardless of execution time.
- **Peripheral I/O Quirks:** Getting the PS/2 keyboard to work required filtering out break codes and mapping raw hex values instead of standard ASCII. Similarly, the push buttons did not behave exactly as expected regarding active low masking, so we had to print the raw register values to correct the bitwise operations.

8 Work Partitioning and Time Log

We used a GitHub repository to manage version control and track issues. We started with pair programming to draft the conceptual architecture and agree on the FreeRTOS primitives. After that, we branched out to work independently on our assigned modules, communicating regularly. We pair programmed again at the end to merge the branches and debug the final integrated system. The project was completed over the course of approximately two weeks.

Team Member	Assigned Tasks & Contributions	Estimated Time
Christian Motta	Hardware ISRs, push button/switch debouncing, PS/2 keyboard processing, and developing the VGA display engine/UI.	25 Hours
Tania Pan	Frequency and RoC calculations, load management state machine, 500ms software timers, and shedding/reconnecting logic.	25 Hours

9 Conceptual Diagram

